

数据读取

数据读取

- 1.快速上手
- 2.硬件接线
- 3.使用方法
- 4.现象结果
- 5.代码解释

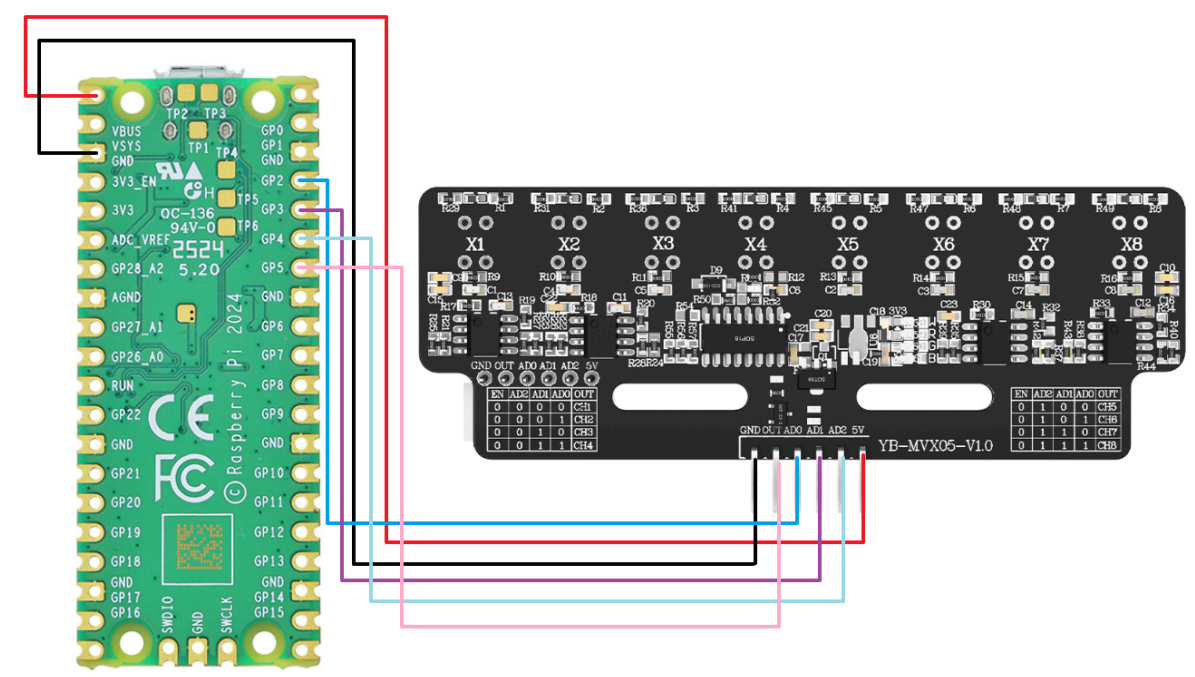
1.快速上手

本教程讲解如何使用PICO2主板读取八路灰度巡线模块的数字量信息，并通过Sheel窗口查看打印结果。

教程中使用的硬件是亚博售卖的PICO2、八路灰度巡线模块，查看下方的接线图进行接线后，再给PICO2烧录程序，就可以获得数据了。非亚博的模块仅供参考。

2.硬件接线

PICO2的microusb口需要使用microusb线连接USB口



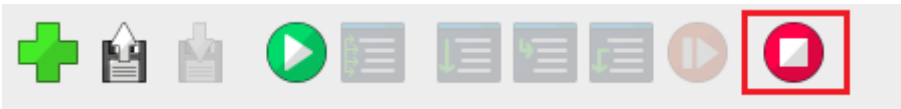
八路灰度巡线模块	PICO2
5V	VBUS
GND	GND
AD0	GP2
AD1	GP3
AD2	GP4
OUT	GP5

亚博售卖的八路灰度巡线模块标配的线材为XH2.54转6pin杜邦线，XH2.54排线接口一端要接在八路灰度模块上，另一端的杜邦线则与上图一样正常连接即可：

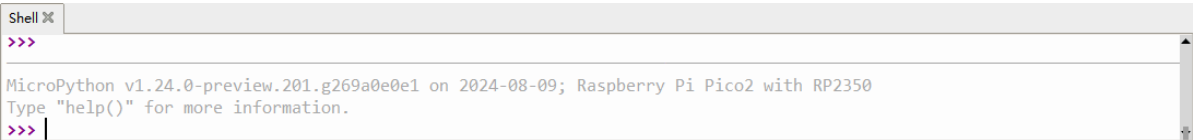


3.使用方法

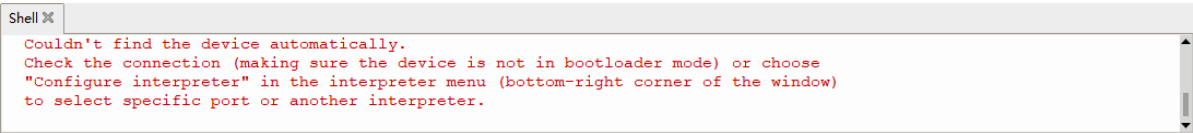
找到代码 Grayscale_Read ，使用Thonny软件打开，接入树莓派pico2后，点击上方工具栏最右侧的停止键：



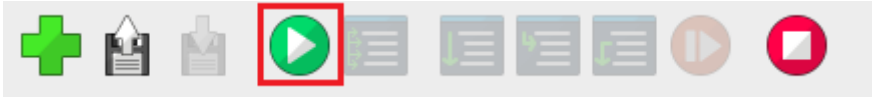
然后可以看见最下方的消息栏中弹出pico2中当前固件的信息，这样代表软件已经识别到了串口：



如果这里的消息栏显示无法识别串口，或者不会基础操作使用的用户，则需要查阅PICO2主板的资料，找到开发环境搭建（python）的教程，学习相关的基础使用以及烧录固件。给主板刷入固件就能识别串口了：



成功识别串口并有固件信息打印后，就点击绿色的按钮开始运行脚本：



4.现象结果

在Thonny中运行程序后，可以看到下方的Shell窗口持续在打印八路灰度巡线模块的数字量，S1对应的是模块上面的X1，当X1的灯亮起的时候，S1的值就为1。

```
[S1:0] [S2:0] [S3:0] [S4:1] [S5:1] [S6:0] [S7:0] [S8:0]
[S1:0] [S2:0] [S3:0] [S4:1] [S5:1] [S6:0] [S7:0] [S8:0]
[S1:0] [S2:0] [S3:0] [S4:1] [S5:1] [S6:0] [S7:0] [S8:0]
[S1:0] [S2:0] [S3:0] [S4:1] [S5:1] [S6:0] [S7:0] [S8:0]
[S1:0] [S2:0] [S3:0] [S4:1] [S5:1] [S6:0] [S7:0] [S8:0]
[S1:0] [S2:0] [S3:0] [S4:1] [S5:1] [S6:0] [S7:0] [S8:0]
[S1:0] [S2:0] [S3:0] [S4:1] [S5:1] [S6:0] [S7:0] [S8:0]
[S1:0] [S2:0] [S3:0] [S4:1] [S5:1] [S6:0] [S7:0] [S8:0]
[S1:0] [S2:0] [S3:0] [S4:1] [S5:1] [S6:0] [S7:0] [S8:0]
[S1:0] [S2:0] [S3:0] [S4:1] [S5:1] [S6:0] [S7:0] [S8:0]
```

5.代码解释

```

def _select_channel(self, channel):
    """
    选择传感器通道
    Select the sensor channel
    """
    self.ad0.value((channel >> 0) & 0x01)
    self.ad1.value((channel >> 1) & 0x01)
    self.ad2.value((channel >> 2) & 0x01)

def _read_out_value(self):
    """
    读取OUT引脚的值
    Read the value from the OUT pin
    """
    return self.sensor_out.value()

def read_all(self):
    """
    读取所有通道
    Read all channels
    """
    values = []
    for i in range(self.channels):
        self._select_channel(i)
        time.sleep_us(50)
        values.append(self._read_out_value())
    return values

def read_single(self, channel):
    """
    读取单个通道
    Read a single channel
    """
    if channel >= self.channels:
        return 0
    self._select_channel(channel)
    time.sleep_us(50)
    return self._read_out_value()

def print_values(self):
    """
    打印所有传感器值
    Print all sensor values
    """
    values = self.read_all()
    for i, value in enumerate(values):
        print(f"[S{i+1}]:{value}]", end=" ")
    print()

```

- `_select_channel` : 根据传入的通道号，通过控制引脚选择对应的传感器通道。
- `_read_out_value` : 读取当前选定传感器通道的数字输出值。
- `read_all` : 遍历并读取所有8个传感器通道的当前状态。
- `read_single` : 读取指定单个传感器通道的当前状态。

- `print_values` : 获取所有通道的传感器值并将其格式化打印到控制台。